

Master Web Intelligence & Science de Données

DÉVELOPPEMENT MOBILE

— Intitulé du Projet —

Application Mobile de Réseau Social

Rapport Frontend — React Native avec Expo

RÉALISÉ PAR

AIT EL MEHDI MEHDI
EL MONTASER FARAHE

ENCADRANT :

PR. ALJ ABDELKAMEL
Professeur

Remerciements

Nous tenons à remercier chaleureusement notre professeur **M. Alj Abdelkamel** pour son encadrement bienveillant, ses conseils avisés et sa disponibilité tout au long de ce projet. Son accompagnement pédagogique nous a permis d’approfondir nos compétences en développement mobile et d’adopter les meilleures pratiques architecturales. Nos remerciements s’adressent également à la Faculté des Sciences Dhar El Mahraz (FSDM) et à l’Université Sidi Mohamed Ben Abdellah (USMBA) pour la qualité de leur formation.

Table des matières

Remerciements	i
Table des matières	1
I Présentation du Projet	3
I.1 Contexte et Objectifs	3
I.2 L'Application Sharely	3
I.3 Périmètre du Rapport	3
II Architecture et Navigation	5
II.1 Fondations Technologiques	5
II.2 Organisation du Code Source	5
II.2.1 Patron deux fichiers	5
II.3 Navigation avec Expo Router	5
II.3.1 Garde d'authentification	6
II.4 Chaîne des Providers	6
II.5 Composants Globaux	6
II.6 Principes Architecturaux	6
III Écrans Principaux et Fonctionnalités	7
III.1 Authentification	7
III.2 Fil d'Actualité	7
III.3 Exploration et Recherche	7
III.4 Reels	7
III.5 Messagerie Directe	8
III.5.1 Liste des conversations	8
III.5.2 Écran de conversation	8
III.6 Profil Utilisateur	8
III.6.1 Profil personnel	8
III.6.2 Profil d'un tiers	8
III.7 Création de Contenu	8
IV UI/UX et Gestion d'État	9
IV.1 Système de Thèmes	9
IV.2 Animations et Interactions	9

IV.3 Gestion d'État par Contexte React	9
IV.3.1 AuthContext	10
IV.3.2 UnreadContext	10
IV.3.3 ThemeContext	10
IV.4 Couche API REST	10
IV.5 Communication Temps Réel (Socket.io)	10
IV.6 Notifications Push	10
V Technologies, Défis et Solutions	11
V.1 Technologies Utilisées	11
V.1.1 Justification des choix clés	11
V.2 Défis Rencontrés et Solutions	12
V.2.1 Persistance des connexions WebSocket lors des navigations	12
V.2.2 Performance du fil d'actualité	12
V.2.3 Superpositions déplaçables sur la caméra	12
V.2.4 URL d'API selon la plateforme	12
V.2.5 Gestion des jetons JWT expirés	12
Conclusion	13

Chapitre I

Présentation du Projet

I.1 Contexte et Objectifs

Ce projet a été réalisé dans le cadre du module *Développement Mobile* de la deuxième année du Master **Web Intelligence et Science des Données (WISD)** à la Faculté des Sciences Dhar El Mahraz (FSDM), Université Sidi Mohamed Ben Abdellah de Fès. L'objectif est de maîtriser le cycle complet de développement d'une application mobile : conception d'interface, navigation, gestion d'état, intégration API et communication temps réel.

I.2 L'Application Sharely

Sharely est une application mobile de réseau social inspirée d'Instagram, développée en React Native avec Expo. Elle cible les plateformes **iOS**, **Android** et **Web** depuis une base de code unique en TypeScript. Ses fonctionnalités couvrent :

- **Compte utilisateur** : inscription, connexion JWT, édition du profil et gestion de l'avatar.
- **Fil d'actualité** : publications photo, défilement infini par curseur, injection de publicités.
- **Stories** : capture caméra avec filtres, superpositions déplaçables et musique de fond.
- **Reels** : flux vertical de vidéos courtes à lecture automatique (style TikTok).
- **Messagerie directe** : conversations en temps réel via WebSockets (Socket.io).
- **Exploration** : recherche d'utilisateurs avec suivi/désabonnement, grille de découverte.
- **Notifications** : toasts temps réel (J'aime, commentaire, abonnement) et panneau dédié.
- **Assistant IA** : chatbot intégré accessible depuis tout écran via un bouton flottant.

I.3 Périmètre du Rapport

Ce rapport porte **exclusivement sur le frontend** React Native. La partie back-end (NestJS, PostgreSQL) est mentionnée uniquement lorsqu'elle éclaire un choix d'implémentation côté client.

Couche	Technologie	Couvert
Interface mobile	React Native + Expo	Oui
Navigation	Expo Router 6	Oui
État global	Context API (React)	Oui
Temps réel client	Socket.io-client	Oui
API REST serveur	NestJS	Référence seulement
Base de données	PostgreSQL	Non

TABLE I.1 – Périmètre du rapport

Chapitre II

Architecture et Navigation

II.1 Fondations Technologiques

Le frontend repose sur **React Native 0.79** avec **Expo SDK 54** et **TypeScript**. Expo fournit un accès simplifié aux APIs natives (caméra, localisation, notifications) sans code natif Swift/Kotlin. **Expo Router 6** assure une navigation basée sur les fichiers, à la manière de Next.js, éliminant toute configuration manuelle des routes.

II.2 Organisation du Code Source

Répertoire	Rôle
<code>app/</code>	Fichiers de routes (Expo Router) — un fichier = une URL navigable
<code>components/</code>	Logique et rendu effectif de chaque écran
<code>context/</code>	Fournisseurs d'état global (ThemeContext, AuthContext, UnreadContext)
<code>utils/</code>	Couche API REST (<code>api.ts</code>) et gestion des WebSockets (<code>socket.ts</code>)
<code>config/</code>	URL de l'API selon la plateforme (<code>env.ts</code>)
<code>assets/</code>	Images, polices et icônes statiques

TABLE II.1 – Structure des répertoires du frontend

II.2.1 Patron deux fichiers

Chaque écran suit un **patron deux fichiers** : `app/Profile.tsx` est un wrapper minimal qui importe et exporte le composant `components/Profile.tsx`, lequel contient toute la logique, l'état et le rendu. Cette séparation maintient les routes propres et rend les composants testables indépendamment.

II.3 Navigation avec Expo Router

Le fichier `app/_layout.tsx` définit la racine de l'application. Il configure une barre d'onglets avec cinq destinations principales et plusieurs routes cachées ou dynamiques.

Route	Composant	Rôle
<code>index</code>	<code>InstagramHomeScreen</code>	Fil d'actualité
<code>explore</code>	<code>ExploreScreen</code>	Recherche et découverte
<code>Reels</code>	<code>ReelsScreen</code>	Vidéos courtes
<code>DirectMessages</code>	<code>DirectMessages</code>	Messagerie
<code>Profile</code>	<code>Profile</code>	Profil personnel
<code>user/[username]</code>	<code>UserProfile</code>	Profil d'un tiers
<code>story/[userId]</code>	<code>StoryViewer</code>	Visualiseur story
<code>post/[id]</code>	<code>PostDetail</code>	Détail publication
<code>conversation/[id]</code>	<code>ConversationScreen</code>	Chat 1-à-1
<code>Camera, CreatePost, CreateReel</code>	—	Création de contenu

TABLE II.2 – Routes de l'application

II.3.1 Garde d'authentification

Le composant `AuthGuard`, monté dans le layout racine, surveille l'état `user` du contexte d'authentification. Tout utilisateur non connecté est redirigé vers `/Auth` ; toute tentative d'accès à `/Auth` par un utilisateur connecté le renvoie vers l'accueil.

II.4 Chaîne des Providers

Les contextes sont imbriqués dans un ordre précis :

`ThemeProvider` → `AuthProvider` → `UnreadProvider` → `AuthGuard` → `TabsLayout`

Cette hiérarchie garantit que chaque provider peut accéder aux contextes dont il dépend — `UnreadProvider` utilise l'identifiant utilisateur fourni par `AuthProvider` pour initialiser ses connexions `Socket.io`.

II.5 Composants Globaux

Trois composants sont rendus au-dessus de tous les écrans via le layout racine :

- `MessageToast` — toast pour les messages directs entrants (auto-dismissal 4 s).
- `NotificationToast` — toast pour les J'aime, commentaires et nouveaux abonnés.
- `ChatBotWidget` — bouton d'action flottant donnant accès à l'assistant IA.

II.6 Principes Architecturaux

Optimistic UI. Les actions (J'aime, abonnement, sauvegarde) modifient l'état local immédiatement avant la confirmation API, offrant une réactivité perçue optimale.

Pagination par curseur. Le curseur de pagination est stocké dans une `useRef` (non réactive) pour éviter les re-rendus inutiles lors des chargements successifs.

Singletons `WebSocket`. Les connexions `Socket.io` sont gérées comme des singletons dans `utils/socket.ts` : une seule instance par namespace, réutilisée lors de toutes les navigations.

Chapitre III

Écrans Principaux et Fonctionnalités

III.1 Authentification

L'écran `AuthScreen` propose deux modes (Connexion / Inscription) commutables par onglets. La connexion envoie `POST /auth/login` et stocke le jeton JWT dans `AsyncStorage` ; l'inscription valide côté client (nom d'utilisateur ≥ 3 caractères, mot de passe ≥ 6 caractères, confirmation identique) avant `POST /auth/register`. Un intercepteur 401 global déclenche la déconnexion automatique en cas de jeton expiré.

III.2 Fil d'Actualité

L'`InstagramHomeScreen` combine un bandeau de stories horizontal et un fil de publications en défilement infini.

Stories. La liste est initialisée par `GET /stories`. La première entrée est toujours la story de l'utilisateur courant avec bouton d'ajout.

Publications. Chargées par tranches de dix via `GET /posts/feed?limit=10&cursor={c}`. Le curseur est conservé dans une `ref` pour éviter les re-rendus. Une publicité (`AdCard`) est injectée tous les huit posts depuis un pool chargé en amont.

Interactions. Chaque `PostCard` offre : J'aime (double-tap ou bouton), sauvegarde, commentaires (`CommentsSheet` slide-up avec réponses imbriquées), partage (`ShareSheet`). L'Optimistic UI garantit un retour immédiat.

Mode Discover. Lorsque l'API retourne `feedType: "discover"`, une bannière invite à suivre des comptes.

III.3 Exploration et Recherche

`ExploreScreen` remplit deux rôles :

- **Recherche d'utilisateurs** : barre de recherche avec debounce 400 ms, appel `GET /users/search?q={q}`, bouton Suivre/Ne plus suivre avec mise à jour optimiste.
- **Grille de découverte** : affichage des publications récentes en trois colonnes ; tap $\rightarrow$ `post/[id]`.

III.4 Reels

`ReelsScreen` propose un flux vertical plein écran (chargé via `GET /posts/reels`). La vidéo démarre automatiquement dès que le reel est visible et se met en pause au défilement. Un tap bascule lecture/pause. La barre latérale droite regroupe J'aime, commentaire, partage, sauvegarde et accès au profil de l'auteur. Les compteurs sont formatés (1,2k / 3,5M).

III.5 Messagerie Directe

III.5.1 Liste des conversations

`DirectMessages` affiche toutes les conversations avec badge non-lu, dernière message, heure et indicateur en ligne. La section « Actifs maintenant » liste les utilisateurs connectés. Un modal *Composer* permet de démarrer une conversation parmi les abonnements.

III.5.2 Écran de conversation

`ConversationScreen` (route `conversation/[id]`) charge l'historique via `GET /chat/{otherId}/messages` puis reçoit les nouveaux messages en temps réel (Socket.io : émission `sendMessage`, écoute `newMessage`). Les bulles adoptent des couleurs différenciées selon l'expéditeur et le thème actif.

III.6 Profil Utilisateur

III.6.1 Profil personnel

`Profile` affiche avatar, bio, statistiques (publications, abonnés, abonnements) et une grille de publications en trois colonnes. L'utilisateur peut modifier son avatar via la galerie (upload `multipart/form-data` vers `PATCH /users/me/avatar`) ou le supprimer. Un menu de paramètres donne accès à l'édition du profil, aux publications sauvegardées/aimées, à la bascule thème et à la déconnexion.

III.6.2 Profil d'un tiers

`UserProfile` (route `user/[username]`) affiche le profil public en lecture seule avec boutons Suivre/Ne plus suivre et Message (ouvre une conversation).

III.7 Création de Contenu

Camera (Story). Capture via `expo-camera`, choix parmi six filtres couleur (Normal, Warm, Cool, Noir, Fade, Vivid), superpositions emoji/texte/localisation déplaçables par `PanResponder`, musique de fond Bensound. Envoi via `POST /media/story`.

CreatePost. Sélection image (`expo-image-picker`), légende (2200 caractères max), lieu, tags. Upload `POST /media/post` puis `POST /posts`.

CreateReel. Enregistrement vidéo via `expo-av`, envoi similaire à `CreatePost`.

Chapitre IV

UI/UX et Gestion d'État

IV.1 Système de Thèmes

`ThemeContext` fournit à toute l'application un thème **clair/sombre** persisté dans `AsyncStorage (@instagram_theme)`. Il expose `{ colors, isDark, toggleTheme }` via le hook `useTheme()`.

La règle absolue du projet : **aucune valeur hexadécimale n'est codée en dur** dans un composant — toute couleur est lue depuis l'objet `colors`. La seule exception documentée est `ReelsScreen`, qui utilise toujours des couleurs sombres (fond de vidéo plein écran).

Token	Clair	Sombre	Usage
background	#FFFFFF	#000000	Fond global
card	#FAFAFA	#1A1A1A	Cartes, modals
text	#262626	#FAFAFA	Texte principal
primary	#7C3AED	#7C3AED	Accent violet
like	#FF385C	#FF385C	Bouton J'aime
storyRing	#7C3AED	#7C3AED	Anneau story

TABLE IV.1 – Tokens de couleurs (extrait)

IV.2 Animations et Interactions

L'application utilise l'API `Animated` de React Native pour des micro-interactions ciblées :

- **J'aime double-tap** : un cœur apparaît en animation *spring* (scale $0 \rightarrow 1.2 \rightarrow 1$) puis disparaît.
- **Bouton J'aime** : légère animation de mise à l'échelle au clic (retour tactile).
- **Barre de progression story** : `Animated.timing` sur 5 secondes, largeur $0 \rightarrow 100\%$.
- **Modals glissants** : `CommentsSheet` et `ChatBotWidget` glissant depuis le bas (`Animated.timing` sur l'axe Y).
- **Indicateur de frappe** : trois points animés en séquence décalée dans le chatbot.
- **Overlays caméra** : chaque superposition dispose de son propre `PanResponder` qui traduit les événements tactiles en coordonnées (x, y) absolues.

IV.3 Gestion d'État par Contexte React

En l'absence de bibliothèque externe (Redux, Zustand), l'état global est géré par trois contextes React :

IV.3.1 AuthContext

Centralise l'authentification. État : `user {id, username, email, fullName, avatar} | null`. Actions : `login()`, `register()`, `logout()`, `refreshUser()`. Persistance : `AsyncStorage` (clés `@instagram_user` et `@token`). Un intercepteur global dans `utils/api.ts` déclenche `logout()` sur toute réponse 401.

IV.3.2 UnreadContext

Gère les compteurs de non-lus en temps réel. Au montage, il initialise les compteurs par `GET /chat/conversations` et `GET /notifications`, puis établit deux connexions `Socket.io` (namespaces `/chat` et `/notifications`). Les événements `"newMessage"` et `"notification"` incrémentent les compteurs et alimentent les toasts correspondants.

IV.3.3 ThemeContext

Détaillé à la section 5.1.

IV.4 Couche API REST

`utils/api.ts` expose quatre fonctions génériques (`apiGet`, `apiPost`, `apiPatch`, `apiDelete`) avec :

- Injection automatique du jeton JWT (`Authorization: Bearer {token}`) à chaque requête.
- Extraction du message d'erreur depuis le JSON de réponse.
- Intercepteur 401 : suppression du token et déclenchement de la déconnexion.

L'URL de base est définie dans `config/env.ts` selon `Platform.OS` : `localhost:3000/api` pour le Web, `{IP_locale}:3000/api` pour les appareils mobiles.

IV.5 Communication Temps Réel (Socket.io)

`utils/socket.ts` expose deux fonctions singleton :

- `getSocket(userId)` : connexion au namespace `/chat`, émission de `joinRoom` à la connexion.
- `getNotifSocket(userId)` : namespace `/notifications`, émission de `subscribe`.

Le transport est restreint à `WebSocket` pur (sans long-polling) pour minimiser la latence. Les connexions sont initialisées une seule fois au démarrage (dans `UnreadContext`) et réutilisées sans interruption pendant toute la session.

Le flux de la messagerie directe suit : émission `sendMessage` → serveur → réception `newMessage` sur le socket de l'interlocuteur → mise à jour de `UnreadContext` → affichage du `MessageToast`.

IV.6 Notifications Push

Au démarrage, `expo-notifications` enregistre le jeton push Expo et l'envoie au serveur (`PATCH /users/me/push-token`). Un handler de réponse aux notifications push navigue directement vers la conversation concernée lorsque l'utilisateur tape sur une notification reçue en arrière-plan.

Chapitre V

Technologies, Défis et Solutions

V.1 Technologies Utilisées

TABLE V.1: Principales dépendances du frontend

Bibliothèque	Version	Rôle
react-native	0.79.x	Framework mobile cross-platform
react	19.1.0	Bibliothèque UI de base
typescript	~5.x	Typage statique
expo	~54.0	SDK Expo, accès APIs natives
expo-router	~6.0	Routing basé sur les fichiers
expo-camera	~55.0	Capture photo/vidéo (stories)
expo-image-picker	~17.0	Sélection de médias depuis galerie
expo-av	~16.0	Lecture et enregistrement vidéo
expo-location	~19.0	Géolocalisation + géocodage inverse
expo-notifications	—	Notifications push (Expo Push Service)
expo-media-library	~55.0	Accès médiathèque de l'appareil
socket.io-client	~4.8	WebSockets (chat + notifications)
async-storage	2.2.0	Persistance locale (token, thème)
react-native-reanimated	~4.1	Animations sur le UI thread
react-native-gesture-handler	~2.28	Gestion des gestes tactiles
@expo/vector-icons	~14.x	Icônes Ionicons et autres sets

V.1.1 Justification des choix clés

Expo Router vs React Navigation seul. Expo Router ajoute la convention fichier $\rightarrow$ route, rendant l'arbre de navigation immédiatement lisible et éliminant le code de registration manuelle des routes.

Context API vs Redux/Zustand. Pour un projet de cette taille, Redux ajouterait une complexité (actions, reducers, selectors) non justifiée. Context couvre l'ensemble des besoins de partage global.

Socket.io-client vs WebSocket natif. Socket.io offre reconnexion automatique, gestion des namespaces, et compatibilité directe avec le serveur NestJS/Socket.io.

TypeScript. A permis de typer les réponses API (union `FeedItem = Post | AdItem`), détectant des erreurs de cohérence dès la compilation.

V.2 Défis Rencontrés et Solutions

V.2.1 Persistance des connexions WebSocket lors des navigations

Problème. Les connexions Socket.io créées dans un composant d'écran sont détruites lors du démontage de ce composant, entraînant la perte des événements temps réel.

Solution. Les connexions ont été déplacées dans `utils/socket.ts` (singleton) et initialisées au montage de `UnreadContext` (durée de vie = toute la session). Elles ne sont jamais liées à un composant d'écran individuel.

V.2.2 Performance du fil d'actualité

Problème. Le curseur de pagination et le pool de publicités stockés en `useState` déclenchaient des re-rendus complets de la `FlatList` à chaque chargement de page.

Solution. Migration vers `useRef` pour ces données non-réactives. Combinée à l'Optimistic UI et au paramètre `onEndReachedThreshold={0.4}`, cette approche maintient un défilement fluide même sur des fils longs.

V.2.3 Superpositions déplaçables sur la caméra

Problème. Rendre des overlays emoji/texte/localisation librement repositionnables sur une prévisualisation caméra live sans bibliothèque tierce incompatible avec Expo Managed.

Solution. Utilisation de `PanResponder` (API React Native core). Chaque overlay a son propre `PanResponder` qui met à jour ses coordonnées (x, y) dans le tableau d'état `overlays`. Les overlays sont rendus en `position: "absolute"` au-dessus de la vue caméra.

V.2.4 URL d'API selon la plateforme

Problème. L'URL du serveur varie : `localhost` pour le Web/simulateur iOS, `10.0.2.2` pour l'émulateur Android, et l'IP réseau local pour les appareils physiques.

Solution. `config/env.ts` centralise un switch sur `Platform.OS`. Cette localisation unique évite toute duplication d'adresse dans les composants.

V.2.5 Gestion des jetons JWT expirés

Problème. À la réouverture de l'application après une longue absence, le jeton stocké peut être expiré, causant des erreurs 401 sur toutes les requêtes.

Solution. Un intercepteur dans `utils/api.ts` détecte chaque réponse 401 et déclenche `logout()` depuis `AuthContext`, effaçant l'état et redirigeant vers l'écran d'authentification via `AuthGuard`.

Conclusion

Ce projet a permis de développer le frontend complet de **Sharely**, une application mobile de réseau social de qualité professionnelle, en exploitant l'écosystème React Native / Expo SDK 54.

L'architecture retenue — routing par fichiers (Expo Router), état global via Context API, couche API centralisée, WebSockets en singleton — s'est révélée efficace et maintenable pour un projet de cette envergure. Le patron deux fichiers (`app/` → `components/`), la règle des tokens de couleurs, et l'Optimistic UI ont contribué à un code cohérent et à une expérience utilisateur fluide.

Fonctionnalités couvertes : authentification JWT, fil d'actualité infini, stories avec overlays, reels, messagerie temps réel, exploration, notifications et assistant IA intégré.

Perspectives d'amélioration :

- Extraction de la logique métier vers des hooks personnalisés (`useInfiniteFeed`, `useSocket`).
- Mise en place de tests automatisés (Jest + Testing Library).
- Optimisation du chargement des images avec mise en cache (`expo-image`).
- Internationalisation (Arabe, Français, Anglais) via `i18n-js`.
- Audit d'accessibilité (`accessibilityLabel`, `accessibilityRole`).

Ce projet constitue une base technique solide, illustrant la puissance et la flexibilité de React Native pour répondre aux exigences d'une application sociale moderne multi-plateforme.